

PATRÓN DE DISEÑO DE COMPORTAMIENTO

Strategy

Define una familia de algoritmos, coloca cada uno en una clase separada y hace sus objetos intercambiables.



Agenda

01

Propósito

Qué es y para qué sirve el patrón Strategy

03

Solución

Cómo Strategy resuelve el problema

05

Aplicabilidad e Implementación

Cuándo y cómo aplicar el patrón

02

Problema

El escenario que motiva el uso de este patrón

04

Estructura y Pseudocódigo

Diagrama de clases y ejemplo de implementación

06

Pros, Contras y Relaciones

Ventajas, desventajas y vínculos con otros patrones

CAPÍTULO 1

Propósito

Comprendiendo la esencia del patrón Strategy

¿Qué es el patrón Strategy?

Strategy es un **patrón de diseño de comportamiento** que te permite definir una familia de algoritmos, colocar cada uno de ellos en una clase separada y hacer sus objetos intercambiables.

También conocido como **Estrategia**, este patrón permite que el algoritmo varíe independientemente de los clientes que lo utilizan, promoviendo la flexibilidad y la reutilización del código.



CAPÍTULO 2

El Problema

Un caso práctico que ilustra la necesidad del patrón

La aplicación de navegación

Un día decidiste crear una **aplicación de navegación** para viajeros ocasionales. La aplicación giraba alrededor de un bonito mapa que ayudaba a los usuarios a orientarse rápidamente en cualquier ciudad.

Una de las funciones más solicitadas era la **planificación automática de rutas**. Un usuario debía poder introducir una dirección y ver la ruta más rápida a ese destino mostrado en el mapa.



Evolución de las rutas

La primera versión solo generaba rutas sobre **carreteras**. Las personas que viajaban en coche estaban encantadas. Pero no a todo el mundo le gusta conducir durante sus vacaciones.



Rutas en coche

Primera versión: solo carreteras



Rutas a pie

Segunda actualización



Transporte público

Tercera opción añadida

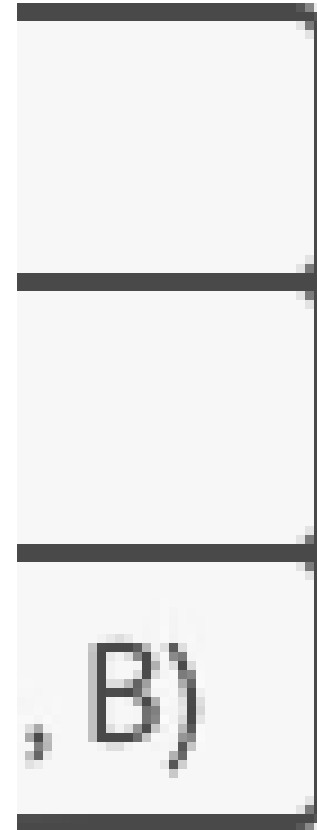


Ciclistas y turismo

Futuras adiciones planeadas

El código del navegador se saturó

Aunque desde una perspectiva comercial la aplicación era un éxito, la parte técnica provocaba muchos dolores de cabeza. **Cada vez que añadías un nuevo algoritmo de enrutamiento, la clase principal del navegador doblaba su tamaño.** En cierto momento, la bestia se volvió demasiado difícil de mantener.



Consecuencias del código saturado



Errores en cascada

Cualquier cambio en alguno de los algoritmos afectaba a toda la clase, aumentando las probabilidades de crear un error en código ya funcional.



Trabajo en equipo ineficiente

Los compañeros dedicaban demasiado tiempo a resolver conflictos de integración al modificar la misma clase enorme.



Difícil mantenimiento

Implementar una nueva función exigía cambiar la misma clase, entrando en conflicto con el código producido por otras personas.

CAPÍTULO 3

La Solución

Cómo el patrón Strategy resuelve el problema

Extraer los algoritmos en clases separadas

El principio clave

El patrón Strategy sugiere que tomes esa clase que hace algo específico de muchas formas diferentes y **extraigas todos esos algoritmos** para colocarlos en clases separadas llamadas **estrategias**.

El Contexto

La clase original, llamada **contexto**, debe tener un campo para almacenar una referencia a una de las estrategias. El contexto delega el trabajo a un objeto de estrategia vinculado en lugar de ejecutarlo por su cuenta.

¿Cómo funciona la delegación?

La clase contexto **no es responsable** de seleccionar un algoritmo adecuado para la tarea. En lugar de eso, el **cliente** pasa la estrategia deseada a la clase contexto.

1

Interfaz genérica

El contexto funciona con todas las estrategias a través de la misma interfaz, que expone un único método para disparar el algoritmo.

2

Independencia

El contexto se vuelve independiente de las estrategias concretas.

3

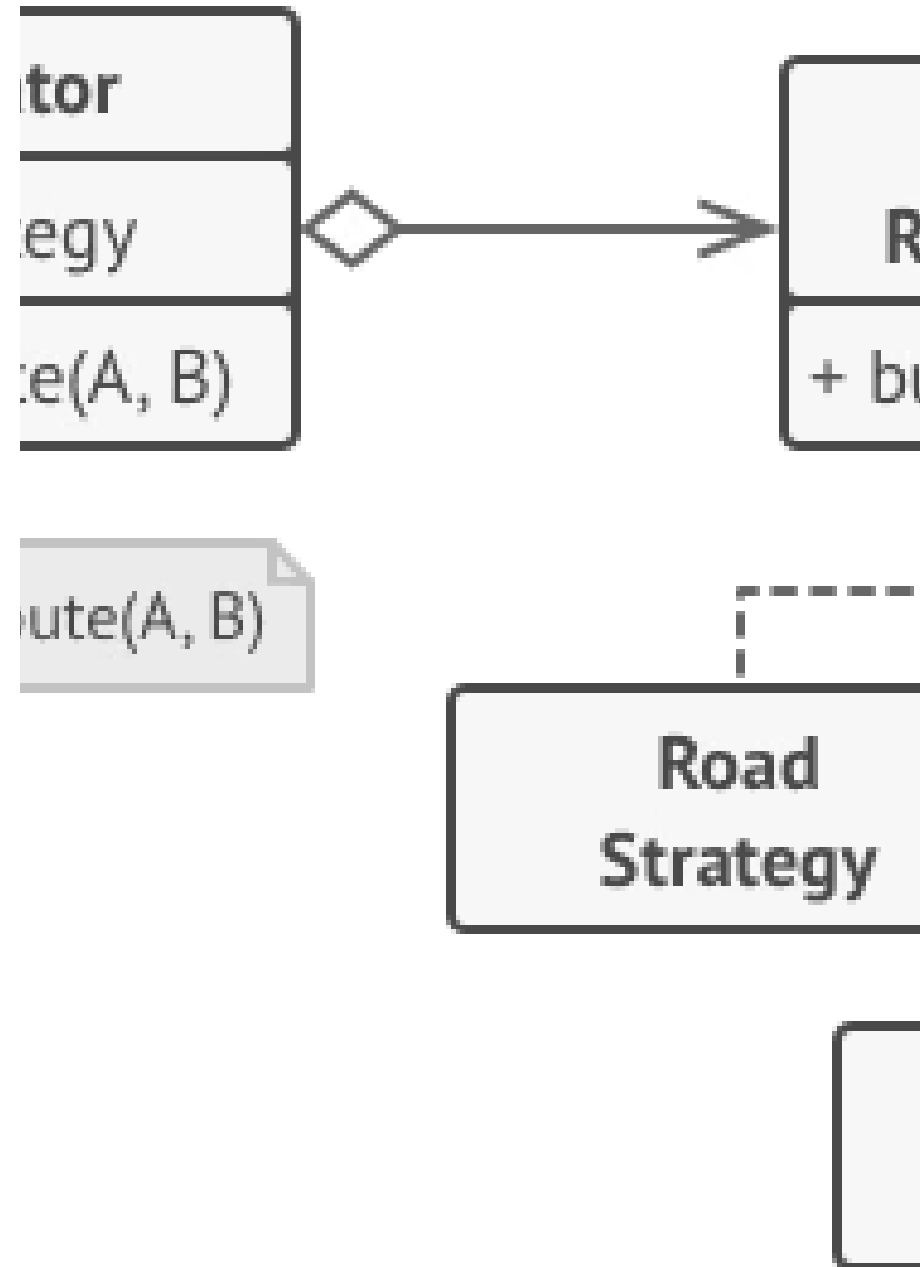
Extensibilidad

Puedes añadir nuevos algoritmos o modificar los existentes sin cambiar el código del contexto ni de otras estrategias.

Estrategias de planificación de rutas

En nuestra aplicación de navegación, cada algoritmo de enrutamiento puede extraerse y ponerse en su propia clase con un único método **crearRuta**. El método acepta un origen y un destino y devuelve una colección de puntos de control de la ruta.

Incluso con los mismos argumentos, cada clase de enrutamiento puede crear una ruta diferente. A la clase navegadora principal **no le importa qué algoritmo se selecciona**, ya que su labor principal es representar un grupo de puntos de control en el mapa.



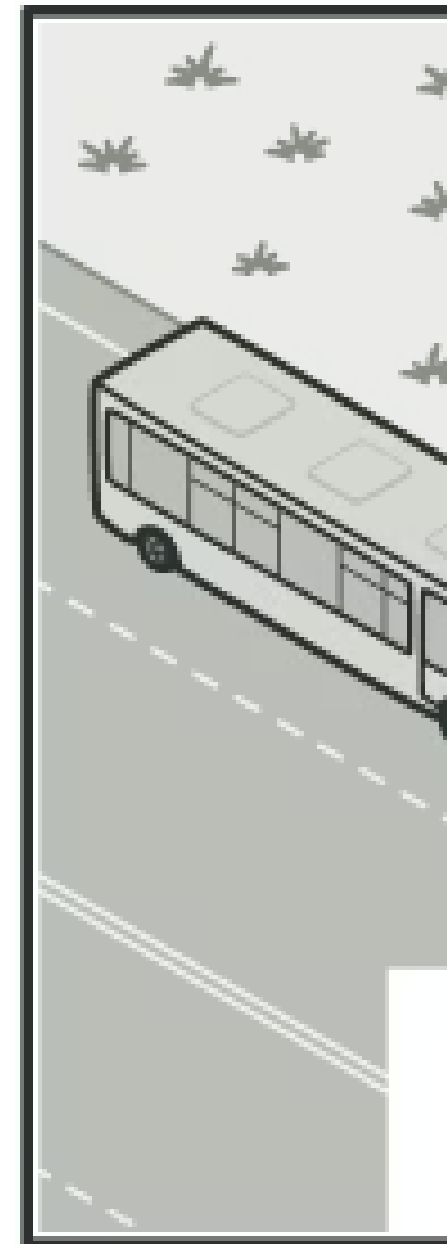
CAPÍTULO 4

Analogía en el Mundo Real

Una forma intuitiva de entender el patrón

Varias estrategias para llegar al aeropuerto

Imagina que tienes que llegar al aeropuerto. Puedes tomar el **autobús**, pedir un **taxi** o ir en **bicicleta**. Éstas son tus estrategias de transporte. Puedes elegir una de las estrategias, dependiendo de factores como el **presupuesto** o los **límites de tiempo**.



CAPÍTULO 5

Estructura

Diagrama de clases del patrón Strategy

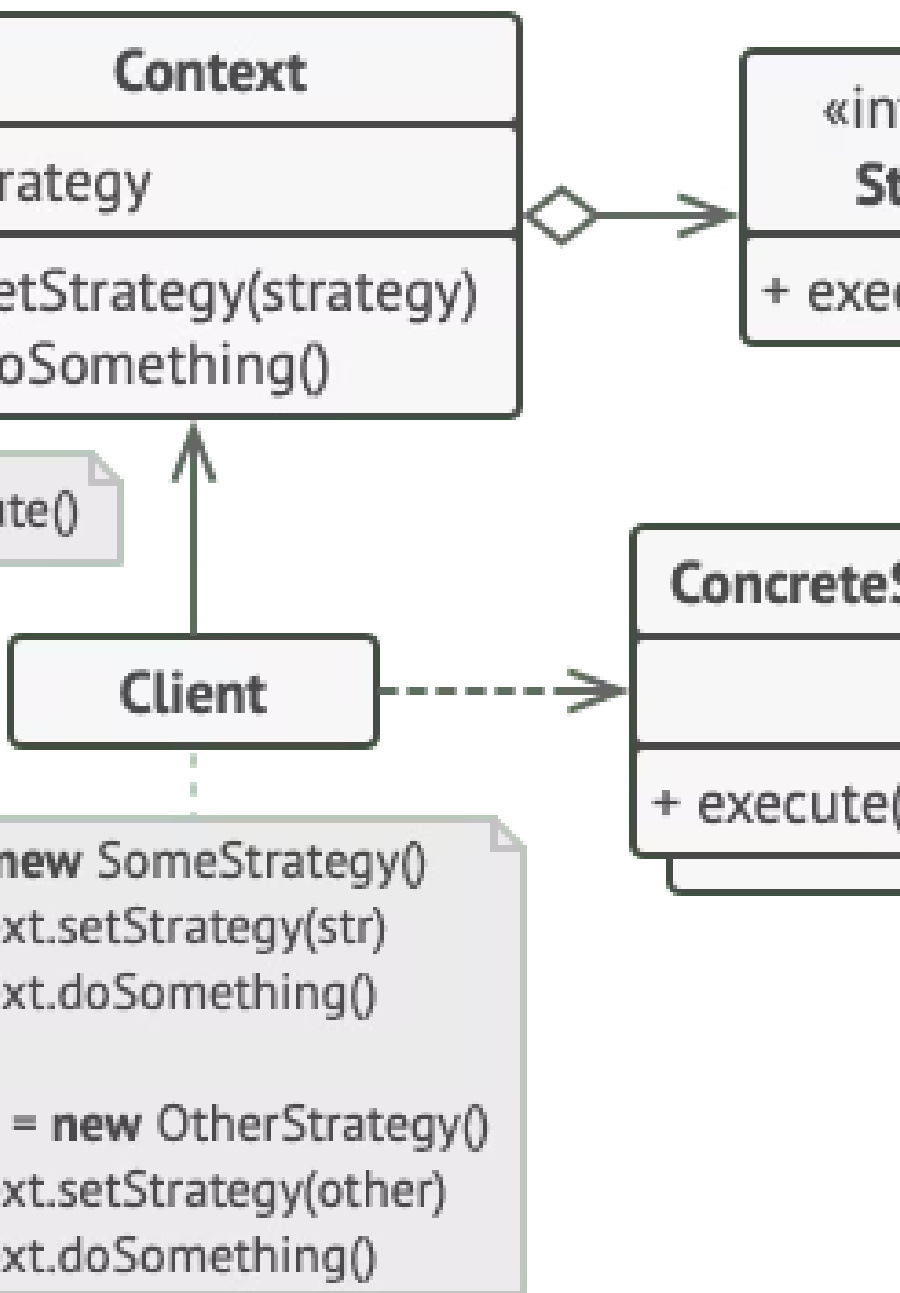


Diagrama de estructura

Componentes de la estructura

1 Contexto

Mantiene una referencia a una de las estrategias concretas y se comunica con este objeto únicamente a través de la interfaz estrategia.

3 Estrategias Concretas

Implementan distintas variaciones de un algoritmo que la clase contexto utiliza.

2 Interfaz Estrategia

Es común a todas las estrategias concretas. Declara un método que la clase contexto utiliza para ejecutar una estrategia.

4 Ejecución y Cliente

El contexto invoca el método de ejecución en el objeto de estrategia vinculado. El cliente crea un objeto de estrategia específico y lo pasa al contexto, pudiendo sustituirlo durante el tiempo de ejecución.

CAPÍTULO 6

Pseudocódigo

Ejemplo de implementación con operaciones aritméticas

Interfaz Strategy y Estrategias Concretas

En este ejemplo, el contexto utiliza varias estrategias para ejecutar diversas operaciones aritméticas.

```
// La interfaz estrategia declara operaciones comunes a todas
// las versiones soportadas de algún algoritmo. El contexto
// utiliza esta interfaz para invocar el algoritmo definido por
// las estrategias concretas.
```

```
interface Strategy is
    method execute(a, b)
```

```
// Las estrategias concretas implementan el algoritmo mientras
// siguen la interfaz estrategia base. La interfaz las hace
// intercambiables en el contexto.
```

```
class ConcreteStrategyAdd implements Strategy is
    method execute(a, b) is
        return a + b
```

```
class ConcreteStrategySubtract implements Strategy is
    method execute(a, b) is
        return a - b
```

```
class ConcreteStrategyMultiply implements Strategy is
    method execute(a, b) is
        return a * b
```

Clase Contexto

```
// El contexto define la interfaz de interés para los clientes.  
class Context is  
    // El contexto mantiene una referencia a uno de los objetos  
    // de estrategia. El contexto no conoce la clase concreta de  
    // una estrategia. Debe trabajar con todas las estrategias a  
    // través de la interfaz estrategia.  
    private strategy: Strategy  
  
    // Normalmente, el contexto acepta una estrategia a través  
    // del constructor y también proporciona un setter  
    // (modificador) para poder cambiar la estrategia durante el  
    // tiempo de ejecución.  
    method setStrategy(Strategy strategy) is  
        this.strategy = strategy  
  
    // El contexto delega parte del trabajo al objeto de  
    // estrategia en lugar de implementar varias versiones del  
    // algoritmo por su cuenta.  
    method executeStrategy(int a, int b) is  
        return strategy.execute(a, b)
```

Código del Cliente

```
// El código cliente elige una estrategia concreta y la pasa al
// contexto. El cliente debe conocer las diferencias entre
// estrategias para elegir la mejor opción.
class ExampleApplication is
method main() is
    Create context object.

    Read first number.
    Read last number.
    Read the desired action from user input.

    if (action == addition) then
        context.setStrategy(new ConcreteStrategyAdd())

    if (action == subtraction) then
        context.setStrategy(new ConcreteStrategySubtract())

    if (action == multiplication) then
        context.setStrategy(new ConcreteStrategyMultiply())

    result = context.executeStrategy(First number, Second number)

    Print result.
```

CAPÍTULO 7

Aplicabilidad

¿Cuándo utilizar el patrón Strategy?

¿Cuándo aplicar Strategy?

1

Variantes de algoritmo en tiempo de ejecución

Cuando quieras usar distintas variantes de un algoritmo dentro de un objeto y poder cambiar de un algoritmo a otro durante el tiempo de ejecución. Strategy te permite alterar indirectamente el comportamiento del objeto asociándolo con distintos subobjetos.

2

Clases similares con comportamiento diferente

Cuando tengas muchas clases similares que sólo se diferencien en la forma en que ejecutan cierto comportamiento. Strategy te permite extraer el comportamiento variante en una jerarquía separada, reduciendo el código duplicado.

3

Aislar lógica de negocio

Para aislar la lógica de negocio de una clase de los detalles de implementación de algoritmos. Los clientes obtienen una interfaz simple para ejecutar los algoritmos y cambiarlos durante el tiempo de ejecución.

4

Eliminar condicionales enormes

Cuando tu clase tenga un enorme operador condicional que cambie entre distintas variantes del mismo algoritmo. Strategy te permite suprimir dicho condicional extrayendo todos los algoritmos en clases separadas.

CAPÍTULO 8

Cómo Implementarlo

Pasos para aplicar el patrón Strategy

Pasos de implementación

Paso 1: Identificar el algoritmo

En la clase contexto, identifica un algoritmo que tienda a sufrir cambios frecuentes. También puede ser un enorme condicional que seleccione y ejecute una variante del mismo algoritmo.

1

2

Paso 2: Declarar la interfaz

Declara la interfaz estrategia común a todas las variantes del algoritmo.

3

Paso 3: Extraer algoritmos

Uno a uno, extrae todos los algoritmos y ponlos en sus propias clases. Todas deben implementar la misma interfaz estrategia.

4

Paso 4: Campo de referencia

En la clase contexto, añade un campo para almacenar una referencia a un objeto de estrategia. Proporciona un modificador **set** para sustituir valores de ese campo. La clase contexto debe trabajar con el objeto de estrategia únicamente a través de la interfaz.

5

Paso 5: Asociar estrategia

Los clientes de la clase contexto deben asociarla con una estrategia adecuada que coincida con la forma en la que esperan que la clase contexto realice su trabajo principal.

CAPÍTULO 9

Pros y Contras

Ventajas y desventajas del patrón Strategy

Ventajas del patrón Strategy



Intercambio en tiempo de ejecución

Puedes intercambiar algoritmos usados dentro de un objeto durante el tiempo de ejecución.



Aislamiento

Puedes aislar los detalles de implementación de un algoritmo del código que lo utiliza.



Composición sobre herencia

Puedes sustituir la herencia por composición.



Principio Abierto/Cerrado

Puedes introducir nuevas estrategias sin tener que cambiar el contexto.

Desventajas del patrón Strategy

Complejidad innecesaria

Si sólo tienes un par de algoritmos que raramente cambian, no hay una razón real para complicar el programa en exceso con nuevas clases e interfaces que vengan con el patrón.

Conocimiento del cliente

Los clientes deben conocer las diferencias entre estrategias para poder seleccionar la adecuada.

Alternativas funcionales

Muchos lenguajes de programación modernos tienen soporte de tipo funcional que te permite implementar distintas versiones de un algoritmo dentro de funciones anónimas, sin saturar tu código con clases e interfaces adicionales.

CAPÍTULO 10

Relaciones con Otros Patrones

Cómo se conecta Strategy con otros patrones de diseño

Strategy, Bridge, State y Adapter

Estructuras similares

Bridge, State, Strategy y, hasta cierto punto, Adapter tienen estructuras muy similares.

Todos estos patrones se basan en la **composición**, que consiste en delegar trabajo a otros objetos. Sin embargo, todos ellos solucionan **problemas diferentes**. Un patrón no es simplemente una receta para estructurar tu código de una forma específica. También puede comunicar a otros desarrolladores el problema que resuelve.

Strategy vs. Command

[Command](#) y [Strategy](#) pueden resultar similares porque puedes usar ambos para parametrizar un objeto con cierta acción. No obstante, tienen propósitos muy diferentes.

Command

Puedes utilizar Command para convertir cualquier operación en un objeto. Los parámetros de la operación se convierten en campos de ese objeto. La conversión te permite **aplazar la ejecución**, ponerla en cola, almacenar el historial de comandos, enviar comandos a servicios remotos, etc.

Strategy

Strategy normalmente describe **distintas formas de hacer lo mismo**, permitiéndote intercambiar estos algoritmos dentro de una única clase contexto.

Más relaciones con otros patrones

Decorator vs. Strategy

[Decorator](#) te permite cambiar la **piel** de un objeto, mientras que [Strategy](#) te permite cambiar sus **entrañas**.

Template Method vs. Strategy

[Template Method](#) se basa en la **herencia**: altera partes de un algoritmo extendiendo esas partes en subclases. [Strategy](#) se basa en la **composición**: altera partes del comportamiento suministrándole distintas estrategias. Template Method trabaja al nivel de la clase (estático). Strategy trabaja al nivel del objeto, permitiéndote cambiar los comportamientos durante el tiempo de ejecución.

State vs. Strategy

[State](#) puede considerarse una extensión de [Strategy](#). Ambos se basan en la composición y cambian el comportamiento del contexto delegando trabajo a objetos ayudantes. Strategy hace que estos objetos sean **completamente independientes**. Sin embargo, State no restringe las dependencias entre estados concretos, permitiéndoles alterar el estado del contexto a voluntad.



Resumen y conclusiones

Define familias de algoritmos

Encapsula cada algoritmo en su propia clase, haciéndolos intercambiables.

Favorece composición sobre herencia

El contexto delega el trabajo sin conocer la implementación concreta.

Principio Abierto/Cerrado

Nuevas estrategias se añaden sin modificar el código existente.

Strategy es uno de los patrones más utilizados en el desarrollo de software moderno. Dominar su aplicación te permitirá escribir código más limpio, flexible y mantenible.